Assignment 3: Database Design Analysis

ProjectRapido – Bike & Cab Booking System
StudentTamajong Bless Ntumnyoh
USN / Roll No.2511021240069
ProgrammeBCA Semester 3 (2025)
CourseDatabase Management Systems
File nameAssignment_3_Database_Design_Analysis.pdf

1. Introduction

This assignment analyses the database design of the proposed Rapido bike and cab booking system. The original design identified seven relations: USER, VEHICLE, CAPTAIN, RIDE, PAYMENT, ZONE and CAPTAIN_ZONE. The analysis below locates remaining redundancy, insertion/deletion/update anomalies, and functional dependencies. It then shows how a poorly designed single-table alternative would fail, and records the improvements that become the **finalized schema** used in Assignments 5–9.

2. Original (logical) schema

```
USER(user_id PK, name, phone_number, email, created_at) VEHICLE(vehicle_id PK, vehicle_type, registration_number, model, capacity) CAPTAIN(captain_id PK, name, phone_number, vehicle_id FK, rating) RIDE(ride_id PK, user_id FK, captain_id FK, pickup_location, drop_location, fare, status) PAYMENT(payment_id PK, ride_id FK, amount, payment_method, payment_status, payment_time) ZONE(zone_id PK, zone_name, city) CAPTAIN_ZONE(captain_id FK, zone_id FK, PRIMARY KEY(captain_id, zone_id))
```

This design is already in a reasonably normalized form (entities separated, M:N handled by a junction table). The remaining risks are missing uniqueness rules, weakly typed attributes, a derived attribute stored without a source table, and free-text locations that can be typed inconsistently.

3. Data redundancy in the original design

1. **Repeated rider and captain identity data if the system were flattened.** If ride history stored rider name and phone beside every ride, the same person would appear many times. The original schema already avoids this by storing identity only in USER and CAPTAIN and referencing them from RIDE.
2. **Fare versus payment amount.** RIDE.fare and PAYMENT.amount represent related money values. If both are stored and not constrained, the same ride can show two different amounts (redundancy plus inconsistency).

3. **Captain rating.** CAPTAIN.rating is a summary of many ride-level opinions. Storing only the average, with no RATING table, means the average cannot be recomputed and can drift from reality.
4. **Pickup and drop as unconstrained text.** Values such as “Koramangala”, “koramangala” and “Koramangala, Bengaluru” can all be entered for the same place. The same locality is then stored many times in slightly different spellings.
5. **No UNIQUE rule on phone, email or vehicle registration.** Two USER rows could hold the same phone number, duplicating one real customer.

4. Insertion, deletion and update anomalies

4.1 How a poor design produces anomalies

Suppose Rapido had been implemented as one unnormalized table:

```
RIDE_BOOKING( ride_id, user_name, user_phone, user_email, captain_name, captain_phone,
captain_rating, vehicle_reg, vehicle_type, vehicle_model, pickup, drop, fare, pay_method,
zone_name, ... )
```

Functional dependencies in that table would include:

- user_phone → user_name, user_email
- captain_phone → captain_name, captain_rating, vehicle_reg
- vehicle_reg → vehicle_type, vehicle_model
- ride_id → all ride facts

Insertion anomaly: a new captain (or a new service zone) cannot be recorded until that captain completes a ride, because there is no place to store captain data except inside a booking row.

Deletion anomaly: if a rider’s only ride row is deleted, the rider’s phone and email disappear with it. The organisation loses a customer record even though the person still exists.

Update anomaly: if Priya Sharma changes her phone number, every historical ride row that copied her phone must be edited. Missing even one row leaves conflicting contact details.

4.2 Anomalies still possible in the original split schema

Anomaly	Where it can still occur	Example in Rapido
Insertion	PAYMENT.ride_id not UNIQUE	Two payment rows could be attached to one ride, breaking the intended 1:1 relationship.

Insertion	No CHECK on status / vehicle_type	A ride could be stored with status “Done” instead of “Completed”, so reports miss it.
Deletion	Missing ON DELETE RESTRICT	Deleting a USER who still has rides would orphan ride history or cascade-wipe bookings.
Update	Stored captain rating	After many new trips, rating may not be updated, so ranking queries are wrong.
Update	fare and amount independent	Ride 13 could show fare 80 while a later “correction” writes amount 70.

5. Functional dependencies

Using the original attribute names, the important FDs are:

Relation	Functional dependencies	Normal form comment
USER	user_id → name, phone_number, email, created_at phone_number → user_id (if unique) email → user_id (if unique)	3NF/BCNF if phone and email are unique candidate keys
VEHICLE	vehicle_id → vehicle_type, registration_number, model, capacity registration_number → vehicle_id	BCNF if registration is unique
CAPTAIN	captain_id → name, phone_number, vehicle_id, rating phone_number → captain_id	3NF; rating is not determined by a ride FD because it is stored as a summary
RIDE	ride_id → user_id, captain_id, pickup_location, drop_location, fare, status	3NF. pickup/drop are not keys of ZONE unless they are constrained to zone names
PAYMENT	payment_id → ride_id, amount, payment_method, payment_status,	BCNF if ride_id is UNIQUE

	payment_time ride_id → payment_id (intended 1:1)	
ZONE	zone_id → zone_name, city (zone_name, city) → zone_id	BCNF with composite uniqueness
CAPTAIN_ZONE	(captain_id, zone_id) is the only key; no non-key attributes	Already 3NF; this is the correct treatment of the M:N FD

There is no non-trivial transitive dependency of the form ride_id → user_id → user_name inside RIDE, because user_name is not stored on RIDE. That is why splitting entities removes the classic 2NF/3NF update problem.

6. Why poor design causes these problems (summary)

A relational schema must store each fact in one place and use keys to point at that fact. Mixing many entities in one table, or leaving uniqueness and domain rules unstated, forces the DBMS to accept duplicate facts. Applications then insert incomplete rows, delete too much, or update only some copies of a fact. Integrity constraints (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, DEFAULT) are the practical way to make the functional dependencies unbreakable.

7. Proposed improvements (finalized design)

1. Rename USER to **users** and CAPTAIN to **captains** (USER is a reserved word in SQL). Use **full_name** instead of name.
2. Add UNIQUE on users.phone_number, users.email, captains.phone_number, vehicles.registration_number, and (zones.zone_name, zones.city).
3. Make payments.ride_id UNIQUE so one ride has at most one payment row.
4. Add CHECK constraints for vehicle_type, capacity, rating, fare, ride status, payment method and payment status.
5. Add DEFAULT values: rides.status = 'Requested', payments.payment_status = 'Pending', vehicles.capacity = 1, captains.rating = 0.00, timestamp defaults.
6. Declare FOREIGN KEY actions: ON UPDATE CASCADE, ON DELETE RESTRICT for operational tables; CASCADE on the junction table captain_zones.
7. Add rides.requested_at so reports can be time-based.
8. Keep pickup_location / drop_location as zone-style names for this project scale, but load them from the same vocabulary as ZONE (Koramangala, Indiranagar, Whitefield, HSR Layout, MG Road) to reduce spelling redundancy.
9. Keep captains.rating as a cached summary for this assignment (with CHECK 0–5), while noting that a future RIDE_RATING table would be the fully normalized source.

The improved relations used from Assignment 5 onwards are:

```
users(user_id PK, full_name, phone_number UNIQUE, email UNIQUE, created_at DEFAULT)
vehicles(vehicle_id PK, vehicle_type CHECK, registration_number UNIQUE, model, capacity
CHECK/DEFAULT) captains(captain_id PK, full_name, phone_number UNIQUE, vehicle_id FK, rating
CHECK/DEFAULT) rides(ride_id PK, user_id FK, captain_id FK, pickup_location, drop_location,
fare CHECK, status CHECK/DEFAULT, requested_at) payments(payment_id PK, ride_id FK UNIQUE,
amount CHECK, payment_method CHECK, payment_status CHECK/DEFAULT, payment_time) zones(zone_id
PK, zone_name, city, UNIQUE(zone_name, city)) captain_zones(captain_id FK, zone_id FK, PRIMARY
KEY(captain_id, zone_id))
```

> These improvements remove the remaining insertion/update holes without changing the business meaning of the original Rapido ER design. Assignments 5–9 implement this finalized schema and prove the constraints with executable SQL.